ELO320 - Estructuras de Datos y Algoritmos
Análisis de Complejidad: F. Recurrencia
Método Maestro

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Funciones de Recurrencia

¿Cómo podemos determinar la complejidad de MergeSort si éste trabaja recursivamente?

- Peor escenario: $\mathcal{O}(n \log n)$
- Mejor escenario: $\Omega(n \log n)$
- Escenario Promedio: $\Theta(n \log n)$

Funciones de recurrencia, relaciones matemáticas recursivas que son utilizadas en ciencias de la computación para explicar la complejidad de éste tipo de algoritmos.

Funciones de Recurrencia

Factorial

```
int factorial(int n){
        if(n>0)
                exit(-1);

        if(n==0)
                return 1;
        return n * factorial(n-1); // 1 + T(n-1)
}
```

Sabemos:

- $T(n) = 1$ (si $n = 0$).
- $T(n) = 1 + T(n-1)$ (si $n > 0$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(1) = n$.
- $T(n) = n \rightarrow \mathcal{O}(n)$.

Funciones de Recurrencia

Factorial

```
int factorial(int n){
        if(n>0)
                exit(-1);

        if(n==0)
                return 1;
        return n * factorial(n-1); // 1 + T(n-1)
}
```

Sabemos:

- $T(n) = 1$ (si $n = 0$).
- $T(n) = 1 + T(n-1)$ (si $n > 0$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(1) = n$.
- $T(n) = n \rightarrow \mathcal{O}(n)$.

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }

    return -1;
}
```

- $T(n) = 1$ (si $n = 1$). $T(n) = 1 + T(n/2)$ (si $n > 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }

    return -1;
}
```

- $T(n) = 1$ (si $n = 1$). $T(n) = 1 + T(n/2)$ (si $n > 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }

    return -1;
}
```

- $T(n) = 1$ (si $n = 1$). $T(n) = 1 + T(n/2)$ (si $n > 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }

    return -1;
}
```

- $T(n) = 1$ (si $n = 1$). $T(n) = 1 + T(n/2)$ (si $n > 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

MergeSort

```
void mergeSort(int arr[], int l, int r){ //l=izq, r=der (indices)
    if (l < r) {
        int m = l + (r - l) / 2; // O(1)

        mergeSort(arr, l, m); // T(n/2)
        mergeSort(arr, m + 1, r); // T(n/2)

        merge(arr, l, m, r); // O(n)
    }
}
```

- $T(n) = 1$ (si $n = 1$).
- $T(n) = n + 2 \times T(n/2)$ (si $n > 1$).
- $T(n) = n(1 + \log n) = n + n \log n$
- $T(n) = \mathcal{O}(n \log n)$

Método Maestro

Desarrolle el árbol de recurrencia para la siguiente función:

$$T(n) = T(n/2) + T(n/4) + n^2$$

- ¿Cuál es la rama más larga? ¿Qué significa?
- ¿Cuál es la rama más corta? ¿Qué significa?

Método Maestro

Técnica para cálculo inmediato de complejidades promedio (Θ) bajo ciertas condiciones basado en el árbol de recurrencia de dichas funciones.

Método Maestro

Sea:

$$T(n) = a \times T\left(\frac{n}{b}\right) + f(n)$$

Con $a \geq 1$, $b > 1$, $f(n) > 0$ para $n \geq n_0$ con $n_0 \geq 0$.

- Largo/Altura: $\log_b n$
- Cantidad hojas: $a^{\log_b n} = n^{\log_b a}$
- Costos: $T(n) = \sum_{j=0}^{log_b(n-1)} a^j f(\frac{n}{b^j}) + \Theta(n^{log_b a})$

Método Maestro

Técnica para cálculo inmediato de complejidades promedio (Θ) bajo ciertas condiciones basado en el árbol de recurrencia de dichas funciones.

Método Maestro

Sea:

$$T(n) = a \times T\left(\frac{n}{b}\right) + f(n)$$

Con $a \geq 1$, $b > 1$, $f(n) > 0$ para $n \geq n_0$ con $n_0 \geq 0$.

- Caso 1:
  Si $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ con $\epsilon > 0 \rightarrow T(n) = \Theta(n^{\log_b a})$.

Note que acá buscamos que $\log_b a$ sea mayor que el grado de complejidad c ($\mathcal{O}(n^c)$).

Método Maestro

Técnica para cálculo inmediato de complejidades promedio (Θ) bajo ciertas condiciones basado en el árbol de recurrencia de dichas funciones.

Método Maestro

Sea:

$$T(n) = a \times T\left(\frac{n}{b}\right) + f(n)$$

Con $a \geq 1$, $b > 1$, $f(n) > 0$ para $n \geq n_0$ con $n_0 \geq 0$.

- Caso 1:
  Si $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ con $\epsilon > 0 \rightarrow T(n) = \Theta(n^{\log_b a})$.

Note que acá buscamos que $\log_b a$ sea mayor que el grado de complejidad c ($\mathcal{O}(n^c)$).

Método Maestro

Técnica para cálculo inmediato de complejidades promedio (Θ) bajo ciertas condiciones basado en el árbol de recurrencia de dichas funciones.

Método Maestro

Sea:

$$T(n) = a \times T\left(\frac{n}{b}\right) + f(n)$$

Con $a \geq 1$, $b > 1$, $f(n) > 0$ para $n \geq n_0$ con $n_0 \geq 0$.

- Caso 2:
  Si $f(n) = \Theta(n^{\log_b a} \log^k n)$ con $k \geq 0 \rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.

Note que acá buscamos que $\log_b a$ sea igual que el grado de complejidad c ($\Theta(n^c)$).

Método Maestro

Técnica para cálculo inmediato de complejidades promedio (Θ) bajo ciertas condiciones basado en el árbol de recurrencia de dichas funciones.

Método Maestro

Sea:

$$T(n) = a \times T\left(\frac{n}{b}\right) + f(n)$$

Con $a \geq 1$, $b > 1$, $f(n) > 0$ para $n \geq n_0$ con $n_0 \geq 0$.

- Caso 2:
  Si $f(n) = \Theta(n^{\log_b a} \log^k n)$ con $k \geq 0 \rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.

Note que acá buscamos que $\log_b a$ sea igual que el grado de complejidad c ($\Theta(n^c)$).

Método Maestro

Técnica para cálculo inmediato de complejidades promedio (Θ) bajo ciertas condiciones basado en el árbol de recurrencia de dichas funciones.

Método Maestro

Sea:

$$T(n) = a \times T\left(\frac{n}{b}\right) + f(n)$$

Con $a \geq 1$, $b > 1$, $f(n) > 0$ para $n \geq n_0$ con $n_0 \geq 0$.

- Caso 3:

  Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ con $\epsilon > 0$ y
  $af(\frac{n}{b}) \leq cf(n)$ con $c < 1$ y $n \to \infty$.
  Luego: $\to T(n) = \Theta(f(n))$

Note que acá buscamos que $\log_b a$ sea menor que el grado de complejidad c ($\Omega(n^c)$).

Método Maestro

Técnica para cálculo inmediato de complejidades promedio (Θ) bajo ciertas condiciones basado en el árbol de recurrencia de dichas funciones.

Método Maestro

Sea:

$$T(n) = a \times T\left(\frac{n}{b}\right) + f(n)$$

Con $a \geq 1$, $b > 1$, $f(n) > 0$ para $n \geq n_0$ con $n_0 \geq 0$.

- Caso 3:

  Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ con $\epsilon > 0$ y
  $af(\frac{n}{b}) \leq cf(n)$ con $c < 1$ y $n \to \infty$.
  Luego: $\to T(n) = \Theta(f(n))$

Note que acá buscamos que $\log_b a$ sea menor que el grado de complejidad c ($\Omega(n^c)$).

Método Maestro

El método maestro busca determinar quien elemento predomina en la complejidad del algoritmo que usa recursividad:

- Caso 1: Trabajo de las hojas es polinomialmente mayor, y por ende, domina la complejidad $n^{\log_b a}$.
- Caso 2: Trabajo entre hojas y raíz es asintóticamente igual, entonces complejidad es `nodo` $\times$ `altura`, vale decir, $f(n) \times \log_n$.
- Caso 3: Trabajo en la raíz es polinomialmente mayor, y por ende, domina la complejidad $f(n)$.

Funciones de Recurrencia: Ejemplos

Desarrolle las siguientes funciones de recurrencia, y determine su desempeño promedio Θ usando el método maestro:

- $T(n) = 9T(n/3) + n$
- $T(n) = 4T(n/2) + n^2$
- $T(n) = 5T(n/5) + n^3$
- $T(n) = T(n/4) + T(n/2) + n^2$ ¿Se puede?
- $T(n) = 2T(n/2) + n$
- $T(n) = 49T(n/7) + 1$
- $T(n) = 2T(n/2) + \frac{n^2}{\log n}$ ¿Es posible?